Google

Orchestrating Big Data Workflows with Google Cloud Composer

pwiech@google.com

“Techniki i Technologie Big Data”, December 6, 2023



Google Cloud

About me



Przemek Więch is a Software Engineer working within the Google Cloud group where he is developing Cloud Composer features.

Przemek has worked at Google since 2011.

Przemek holds a PhD in Computer Science from the Warsaw University of Technology.

Google Cloud

Agenda

- What is Apache Airflow?
- What is Cloud Composer?
- Composer basics
- Interesting features

Proprietary + Confidential

01

Apache Airflow

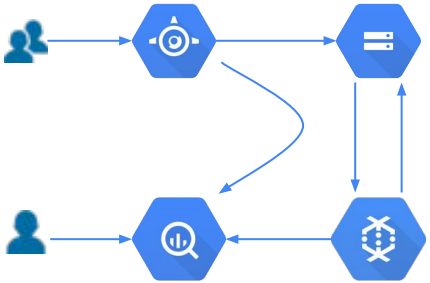
Common Workflow Solutions

Cron simple tasks

```
* * * * * command to run
```

Cost: **low**
Friction: **medium**

Cron + scripting



Cost: **medium**
Friction: **high**

3rd Party Applications

IBM Tivoli software

Informatica

Cost: **high**
Friction: **high**

Google Cloud

main | 91 branches | 4,043 tags | Go to file | Code

potiuk Always bring breeze down befor... | 2375e0d 1 hour ago | 22,256 commits

.devcontainer	Configure vscode dev-container (#26958)	last year
.github	Add 2.8.0b1 to issue template (#35912)	5 hours ago
airflow	Use fail instead of change_state(failed) in K8S e...	6 hours ago
chart	change indent to 4 (#35824)	4 days ago
clients	Update content type validation regexp for go c...	9 months ago
constraints	Remove Python 3.7 support (#30963)	6 months ago
dev	Always bring breeze down before using `--use...`	1 hour ago
docker-context-files	Add capability of iterating locally on constraint ...	8 months ago
docker_tests	Optimize CI/PROD image waiting and verificati...	2 days ago
docs	Update redshift_sql.rst to reference Redshift c...	9 hours ago

About

Apache Airflow - A platform to programmatically author, schedule, and monitor workflows

airflow.apache.org/

python | workflow | airflow | scheduler | apache | apache-airflow

Readme

Apache-2.0 license

Code of conduct

Security policy

Activity

32.5k stars

759 watching

13.2k forks

Apache Airflow

- Apache Airflow is a top-level project in the Apache Software Foundation (ASF)
- Airflow has become very popular within the open-source community
- Designed to enable users to author, schedule and monitor workflows
- Workflows are authored as Directed Acyclic Graphs (DAGs), and can be configured in Python



Why Airflow?

- Better **cron**
- Running **workflows**



Key Airflow concepts

DAGs

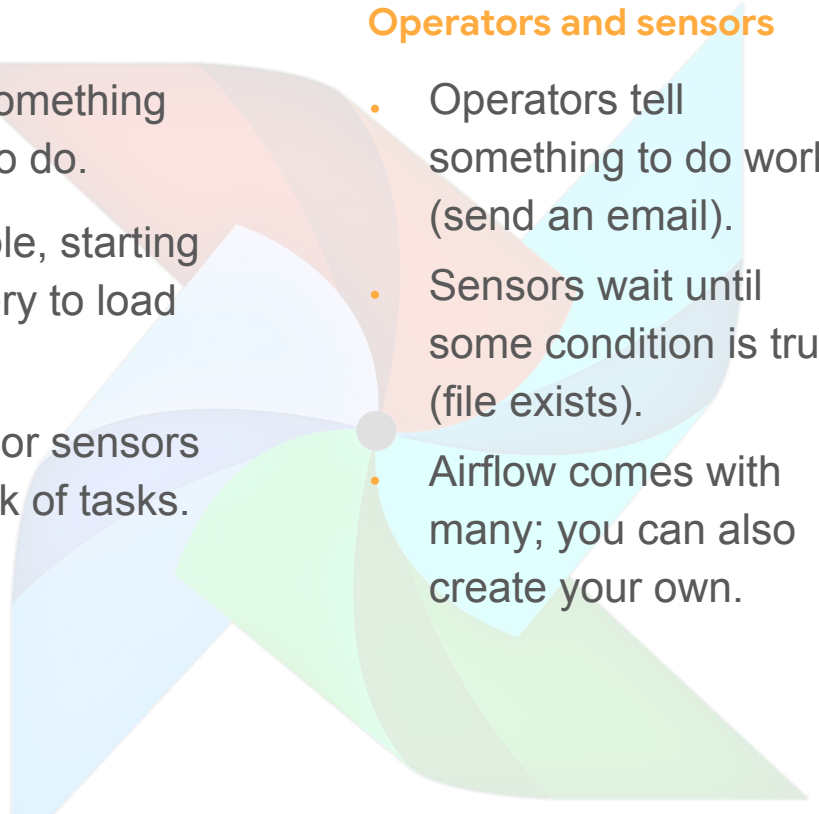
- Every workflow is defined as a directed acyclic graph.
- A DAG defines a series of tasks to do something.
- The tasks can have directional dependencies.

Tasks

- A task is something you want to do.
- For example, starting a SQL query to load data.
- Operators or sensors do the work of tasks.

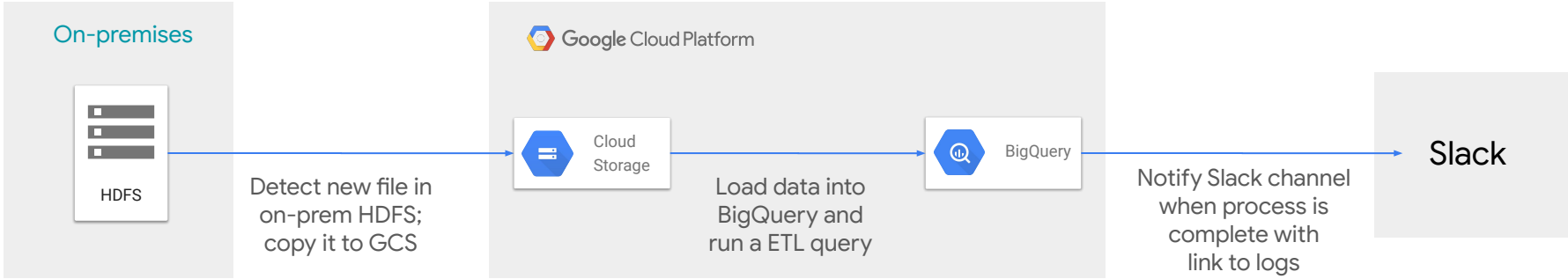
Operators and sensors

- Operators tell something to do work (send an email).
- Sensors wait until some condition is true (file exists).
- Airflow comes with many; you can also create your own.

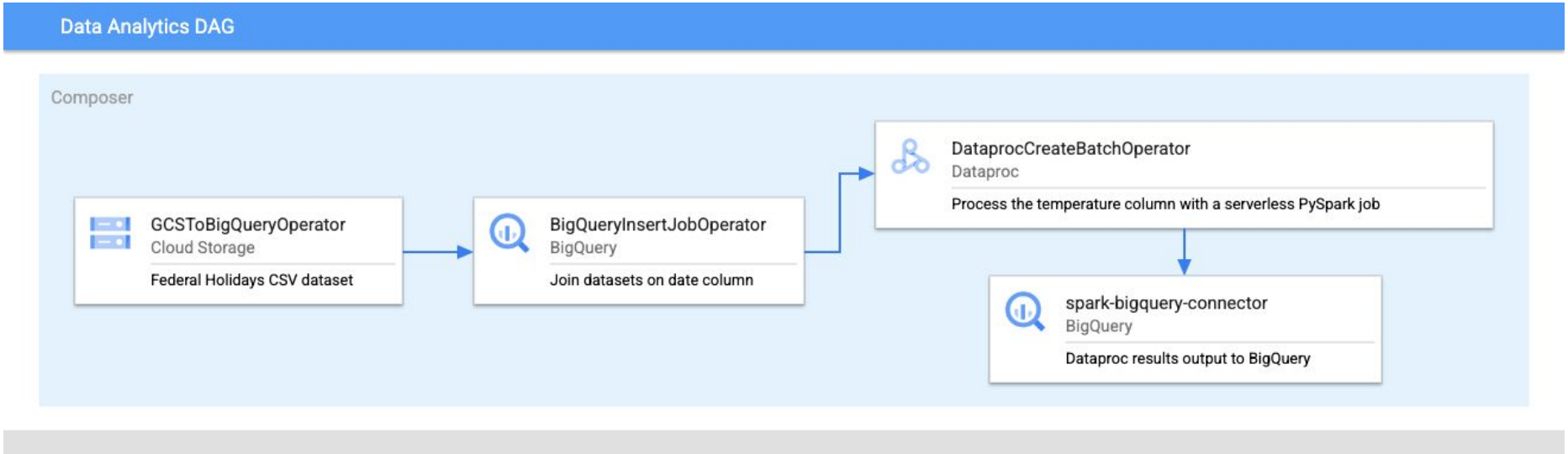


Google Cloud

Example workflow (DAG)



Data analytics workflow



Google Cloud

Additional Airflow functionality

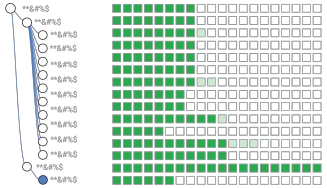


- **Connections -** Store and use connection details to external systems
- **Logging -** Logs for the output of each task in a DAG
- **Pools -** Define parallelism to avoid flooding external systems

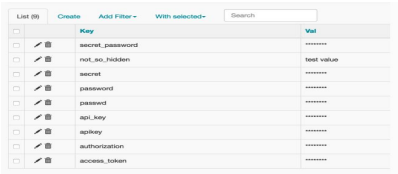
- **SLAs -** Set SLAs on a per-task level and email on SLA failure
- **Variables -** Key-value store shared among all DAGs
- **XComs -** Enables tasks to exchange messages/data

Google Cloud

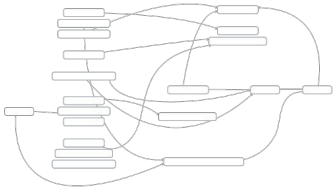
Easier visibility through DAG views



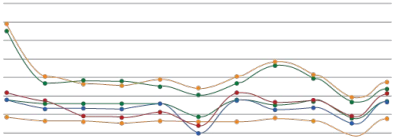
Tree views show a DAG across time, aiding in identifying blockers for late pipelines.



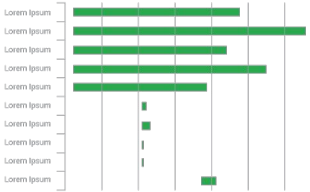
Variable views allow you to inspect the key-value pair of variables used in jobs



Graph views give a comprehensive look at dependencies and status of a work-flow.



Timing charts allow you to analyze task duration



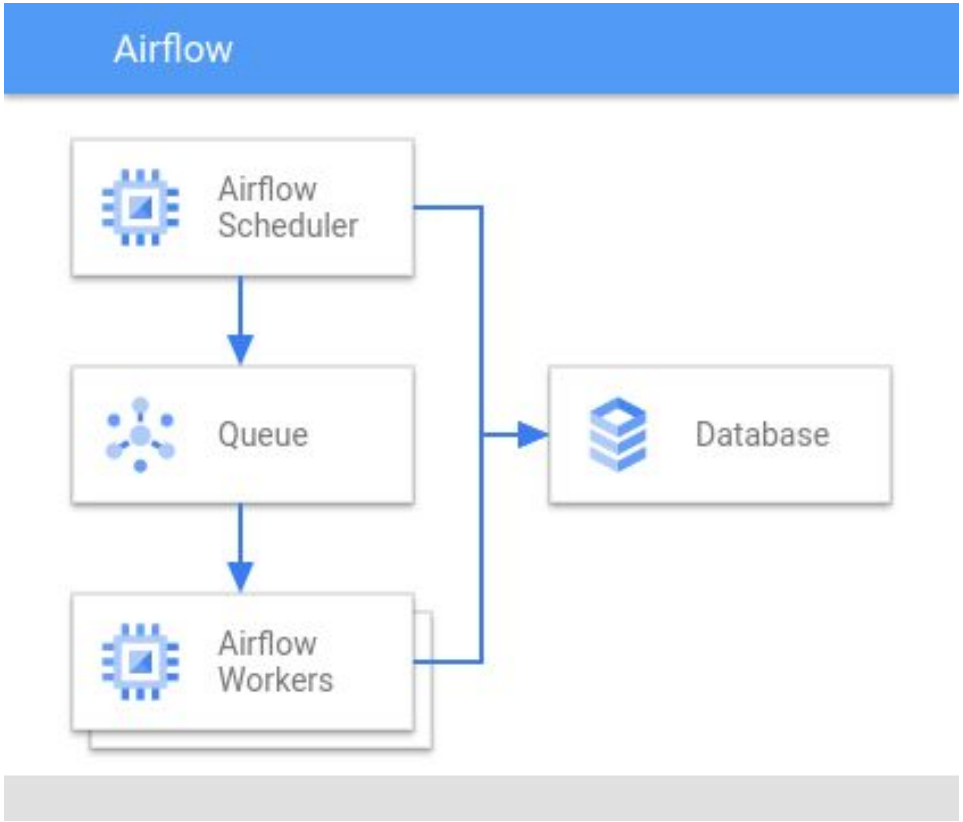
Gantt views allow you to analyze task duration to spot bottlenecks



Code views give you the clearest view of what is happening within your pipeline

Google Cloud

Basic Airflow deployment



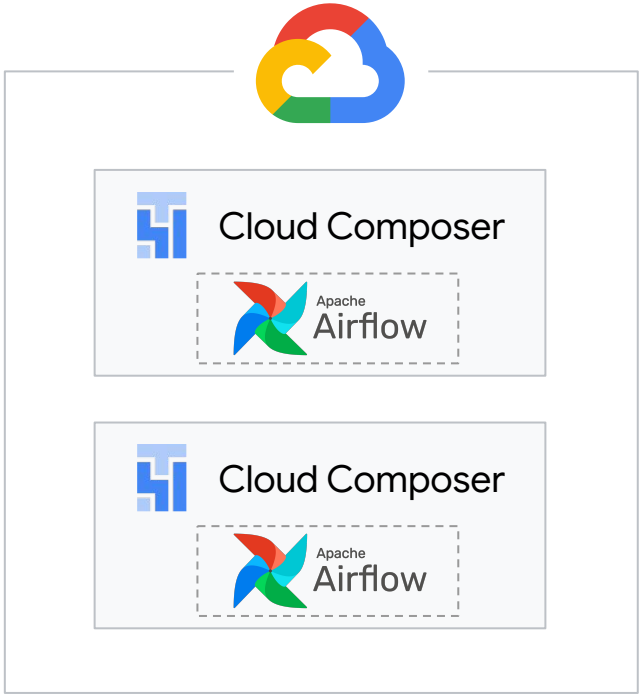
Google Cloud

Proprietary + Confidential

01

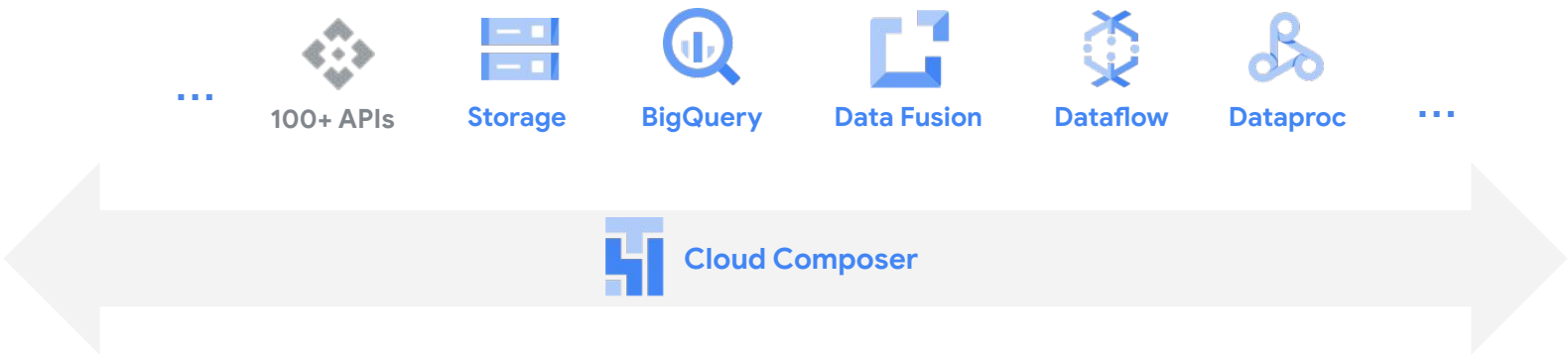
Cloud Composer

Fully managed Apache Airflow environments



Google Cloud

Cloud Composer in a nutshell



*Orchestrate work across **Google Cloud**, external **SaaS services** and **proprietary APIs***

Google Cloud

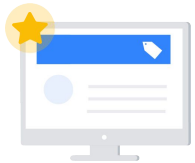
Google Cloud

Cloud Composer environments



- You create a Cloud Composer environment within a project
- Each environment is a managed deployment of Apache Airflow
- Projects can contain more than one environment
- Environments integrate with other GCP services, including:
  - Cloud Storage to manage workflows
  - Cloud Logging to manage logs
  - Cloud Identity and Access Management (IAM)

Cloud Composer benefits



Simple deployment



Enterprise Security Features



Managed infrastructure



Robust Monitoring & Logging



Technical Support



DAG code portability

Google Cloud

Composer = managed Airflow on GCP

Apache Airflow

+ Python/open source
+ Well-established interfaces
  (*CLI / Web server / API server*)
+ Flexible scheduling and dependency management with retries
+ Rich library of connectors
+ Active community support*
- Non-trivial setup & management efforts
- Logging/debugging

+ KubernetesExecutor



Google Cloud Platform

+ Ease of use
  + Automatic DAG synchronization
  + One-click deployment
+ Security
  + Shared VPC
  + Private IP
+ Infrastructure scalability
+ Operability & Maintainability
  + Python dependency management
  + Airflow config update propagation
  + In-place version upgrades
+ Access to logs

- Custom Airflow Packages

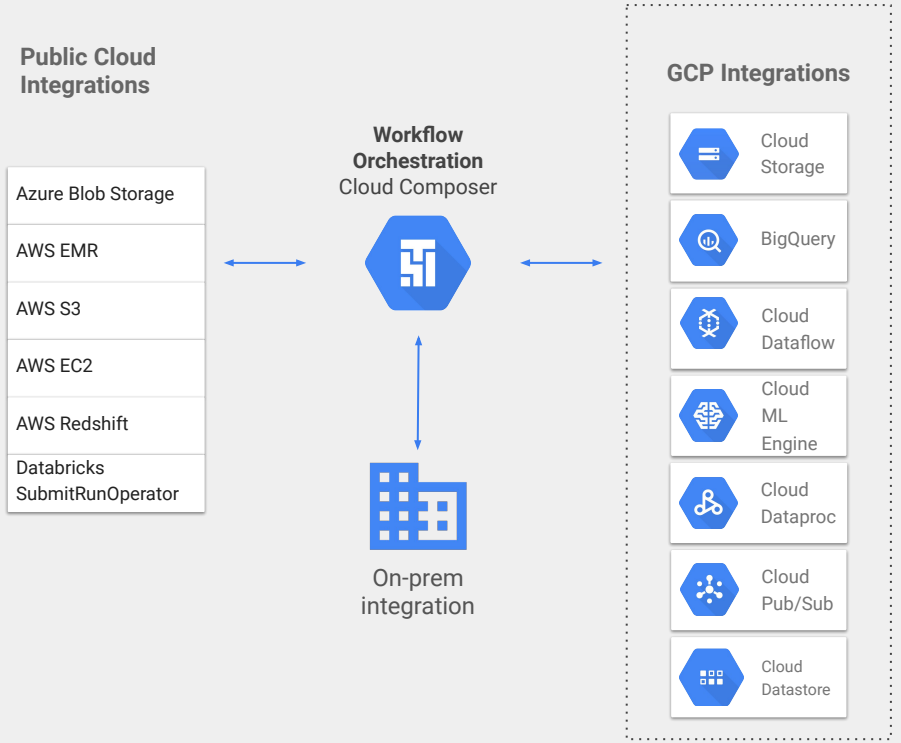
Google Cloud

Rich library of connectors

Broad community of development
Diverse needs of the OSS community ensure that connectors are built to a variety of services, with more added on an ongoing basis

Easy-to-use experience
Drop do-it-yourself orchestration between services that hamper future flexibility by using established connectors

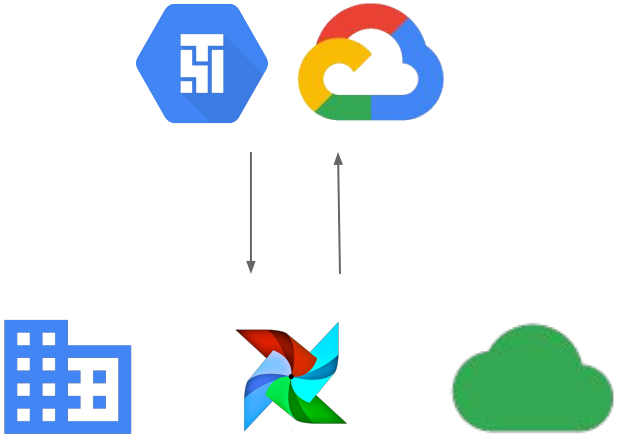
Connect data across environments
Take advantage of what you feel are best in breed solutions from a mix of your on-premises and cloud environments



Google Cloud

OSS: Portability Insurance

Run Cloud Composer in GCP Environment



Run Apache Airflow in on-premises or public cloud environment

Your workflows are as valuable as your data.

Data is only as valuable as the operations that are performed on it to yield information. Workflows represent hours of hard work by data engineers that enable the business to pull the right information, at the right time, to serve its needs.

Keep your workflows portable.

The ability to move your workflows from one platform to another helps ensure that if you do decide to replatform, you can do so at lower cost and with less consumption of engineering resources.

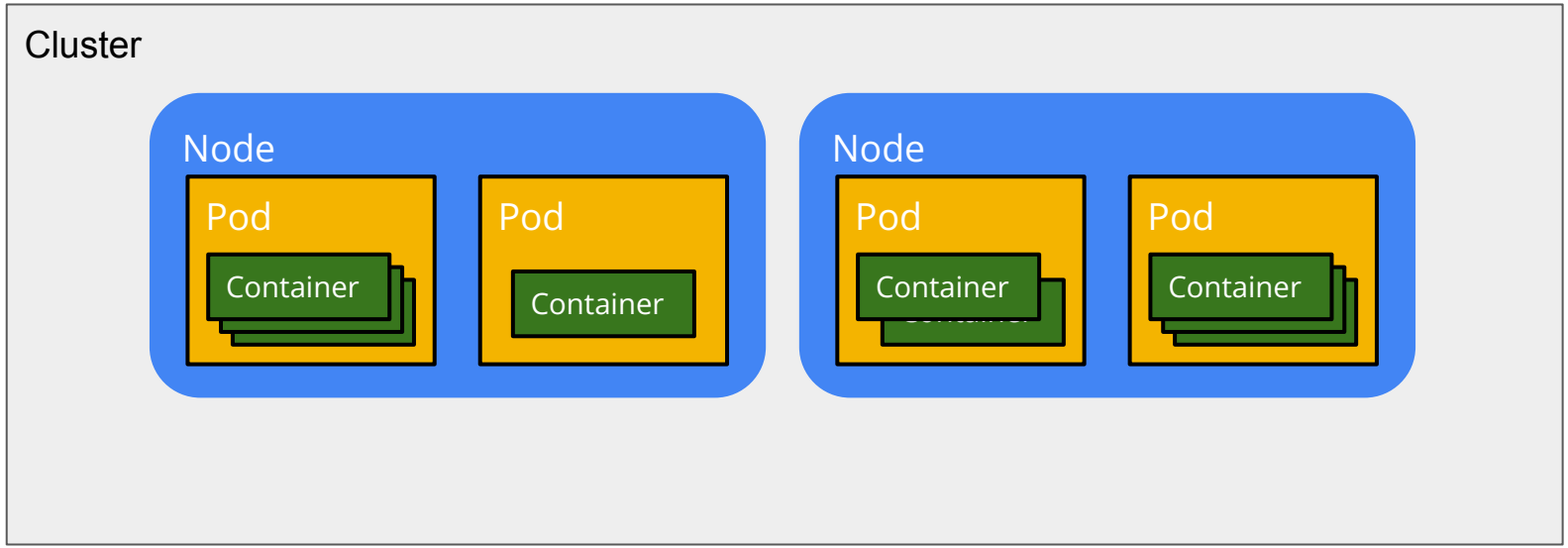
Proprietary + Confidential

01

Composer Architecture

Kubernetes

GKE = Google Kubernetes Engine
Autopilot GKE – adds nodes based on needs



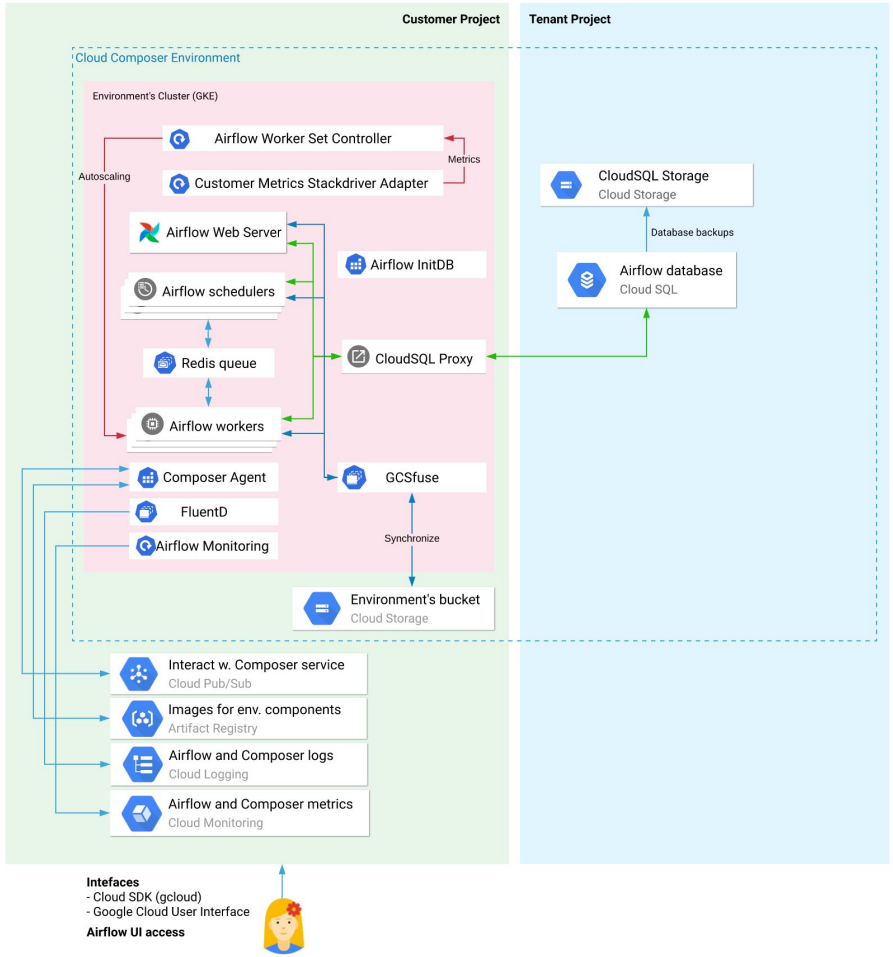
Google Cloud

Cloud Composer 2 architecture

Cloud Composer 2 interacts with the following services:

- **Cloud SQL** - running Airflow metadata storage
- **Cloud Storage** - user uploaded content (DAGs, user data)
- **Kubernetes** - runs Scheduler(s), **Web Server**, Redis queue, SQL proxy and Airflow workloads
- **Cloud Logging** - stores and indexes components logs
- **Cloud Monitoring** - searchable Cloud Composer metrics

... and many more that we are managing for you.



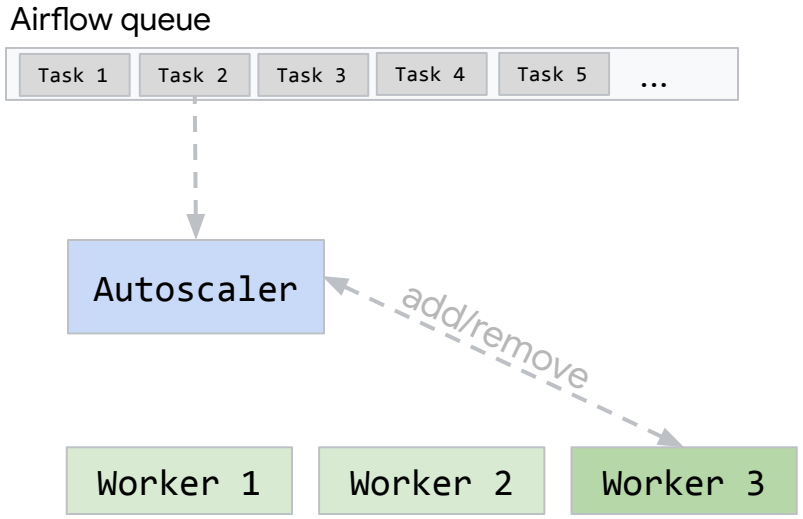
Composer 2 Autoscaling

The challenge

- Sizing environments for variable workloads
- Task failures due to capacity constraints
- Cost control

Efficiency, Scalability, and Simplicity

Autoscaling, and **usage-based pricing** optimizes costs.



Google Cloud

Google Cloud

Creating and accessing environments

Cloud Platform Console

Under the Composer section of the Cloud Platform console

Easily manage environments

Links to the Cloud Storage bucket and the Airflow WebUI for an environment

Google Cloud SDK CLI

Ships with the Google Cloud SDK (gcloud)

Use the command `gcloud composer`

Can be used to issue Airflow CLI commands with `composer environments run`

Terraform

Infrastructure automation software

Declaratively configure Composer environments

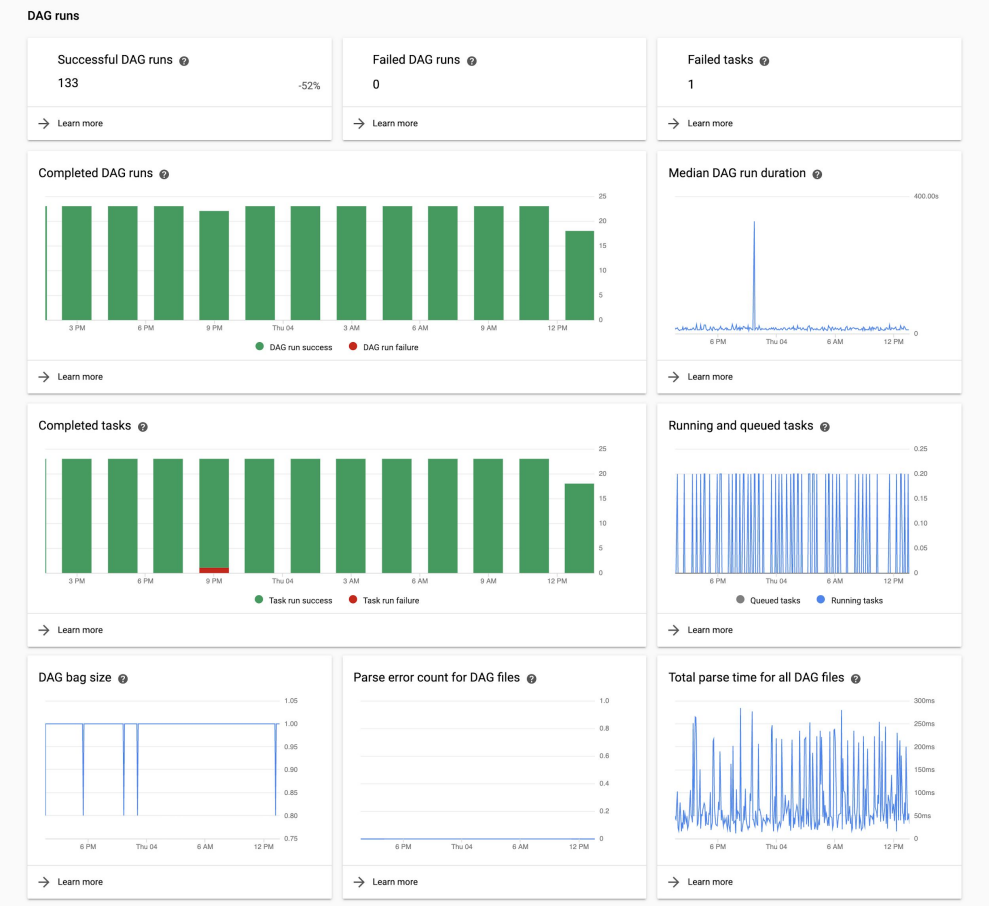
Cloud Composer APIs

REST and RPC APIs for managing environments

Useful for scripting environment management

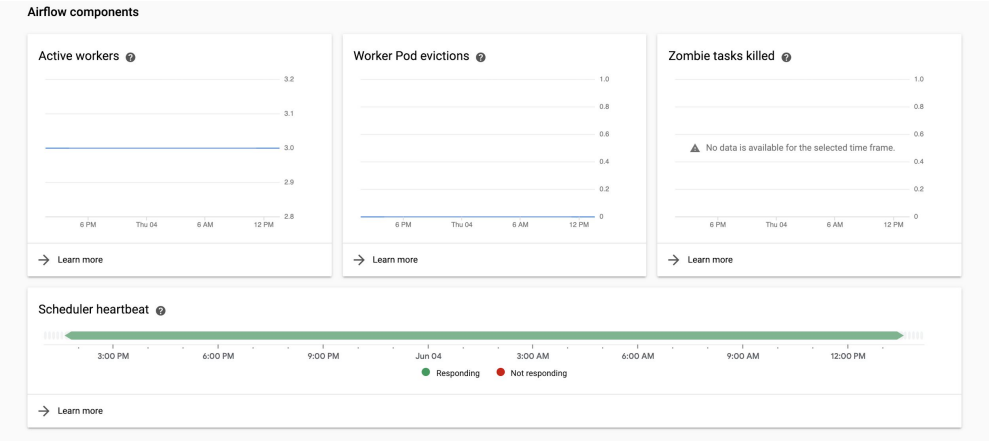


DAG runs monitoring



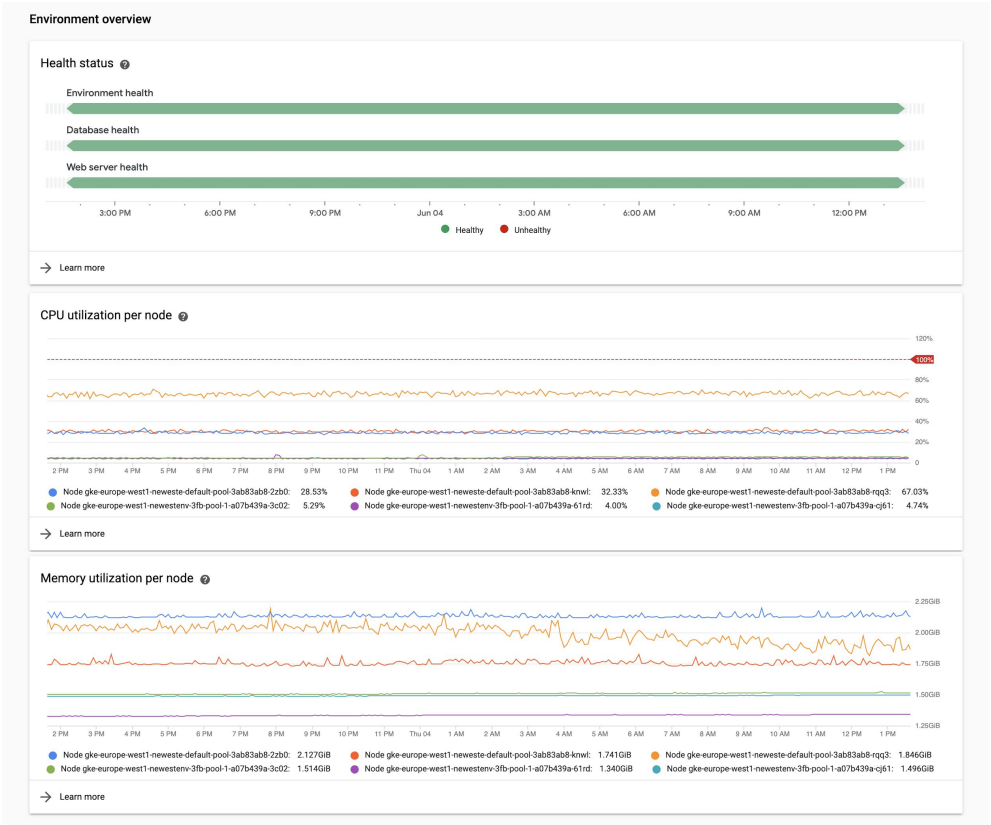
Google Cloud

Airflow Monitoring



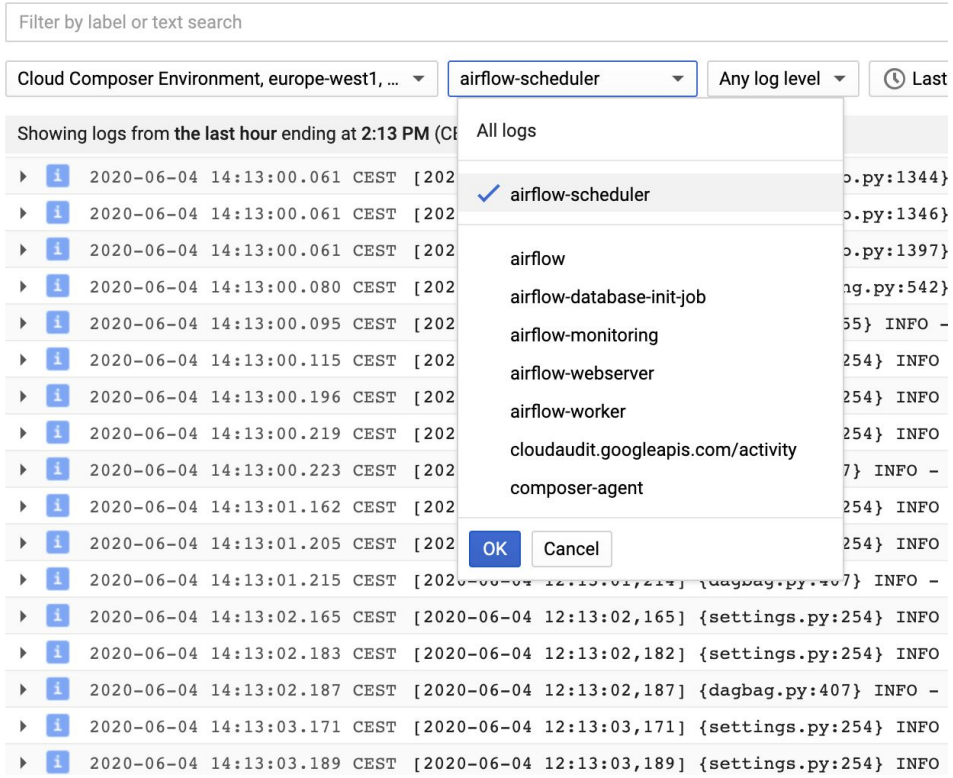
Google Cloud

Infrastructure Monitoring



Google Cloud

Use Composer Logging holistically



Google Cloud

High Availability and Disaster Recovery

HA

Set of measures ensuring uptime and availability. Oriented on avoiding a single-point-of-failure.

MTBF (Mean Time Between Failures): *"Ideally failures will never happen"*

DR

Set of measures ensuring recovery of the system in case of an unplanned outage.

Often a mix of automated, semi-manual, and manual steps.

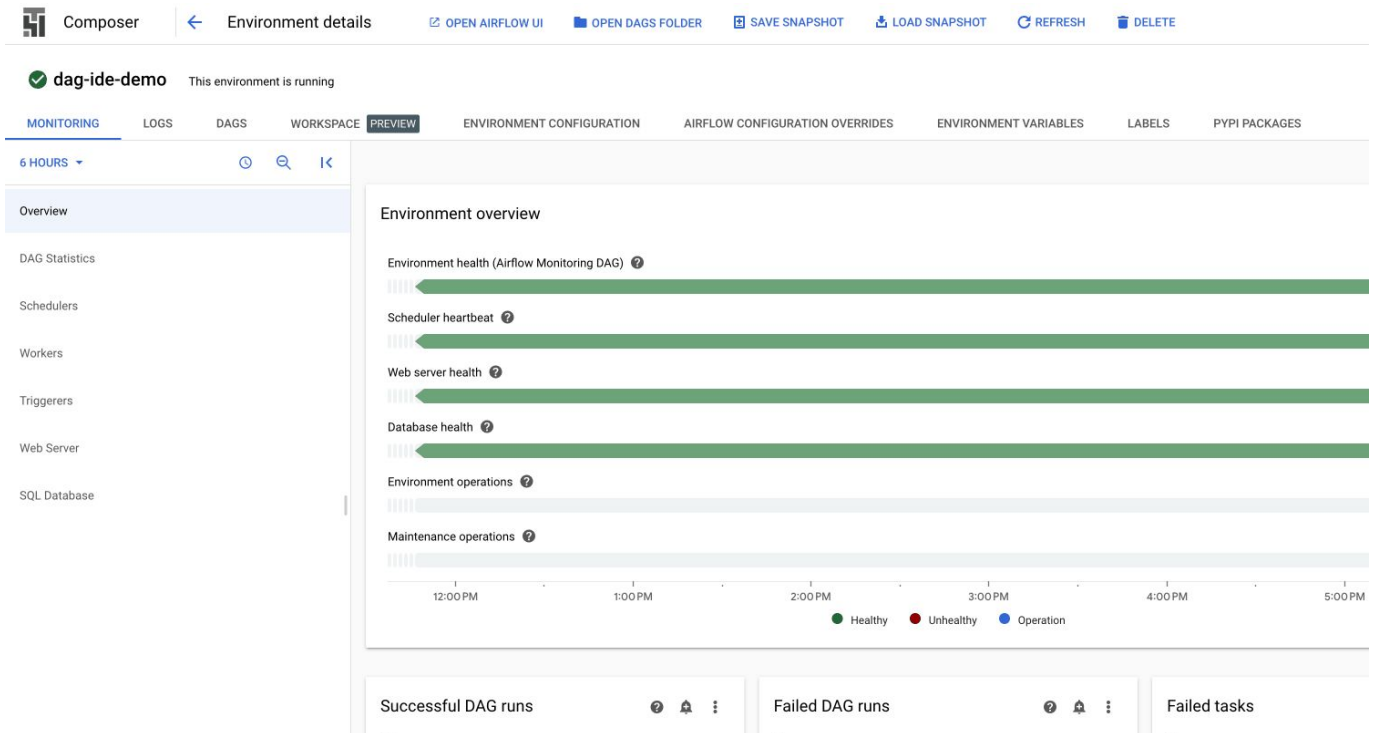
MTTR (Mean Time To Resolution): *"Ideally if any failure does happen, system can recover instantly"*



Proprietary + Confidential

Duet AI in Cloud Composer

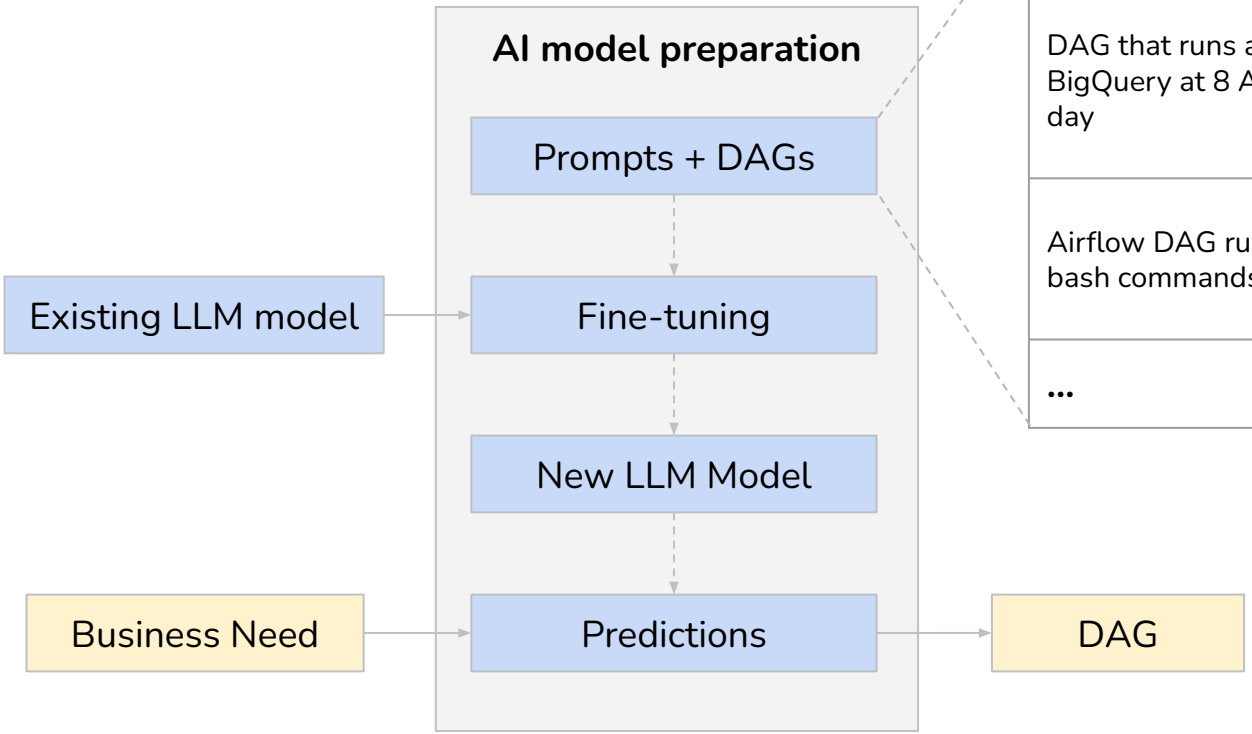
Prompt: *Write Airflow DAG that that runs BigQuery query using BigQueryInsertJobOperator to get number of GitHub commits in May 2022. Please, use public dataset for github data published by Google Cloud Platform*



Google Cloud

Proprietary + Confidential

How to use LLMs?



Prompt	DAG

DAG that runs a query in BigQuery at 8 AM every day	`from airflow import DAG`
	`from airflow.providers.google.cloud.operators.bigquery import BigQueryOper`
	`from airflow.utils.dates import days_ago`
	
	`with DAG(`
	`  "bigquery-query",`
	`  schedule_interval="0 8 * * *",`
	`) as dag:`
	
	`  run_query = BigQueryOperator(`
	`  task_id="run_query",`
	`  sql="SELECT * FROM `my_dataset.my_table`",`
	`  use_legacy_sql=False,`
	`  )`

Airflow DAG running 10 bash commands in parallel	`from airflow import models`
	`from airflow.operators.bash_operator import BashOperator`
	
	`with models.DAG(`
	`  "Bash_10_parallel",`
	`  schedule_interval="0 0 * * *",  # Override to match your needs`
	`) as dag:`
	
	`  for k in range(10):`
	`  newStep = BashOperator(`

...	

Google Cloud

Proprietary + Confidential

Enterprise features

- Customer Managed Encryption Keys
- Virtual Private Cloud Service Controls
- Web server network access control levels
- Secret Manager as a storage for sensitive configuration data
- Audit logging
- Access Transparency
- Data Residency and Location Restrictions
- Data lineage integration with Dataplex

Proprietary + Confidential

01

Creating DAGs

Google

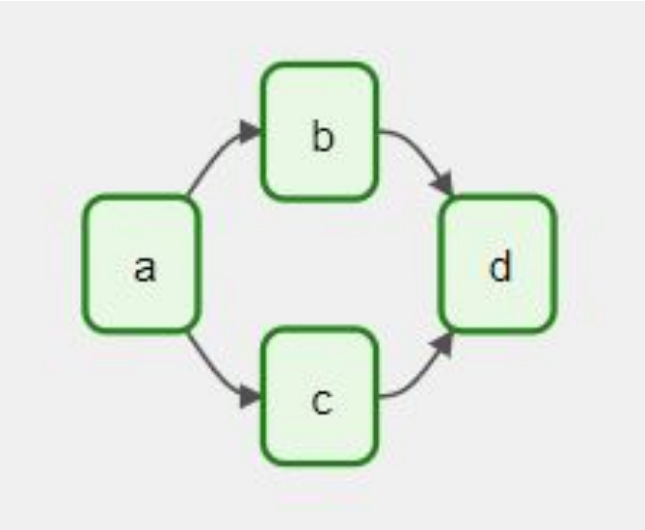
Directed Acyclic Graph (DAG)

DAG is a **collection** of all the **tasks you want to run**, organized in a way that reflects their **relationships and dependencies.**

A DAG is defined in a Python **script**, which represents the **DAG's structure** (tasks and their dependencies) as code.

DAG has the following components:

- DAG definition
- Tasks
- Task Relationships




```
dag = DAG(
    'tutorial',
    default_args=default_args,
    description='A simple tutorial DAG',
    schedule_interval=timedelta(days=1),
)
```

An example DAG

Google Cloud

DAG Runs

A **DAG run** is a **physical instance** of a DAG, containing **task instances** that run for a specific `execution_date`.

A DAG run is usually created by the Airflow **scheduler**, but can also be created by an external trigger. Multiple DAG runs may be running at once for a particular DAG, each of them having a different `execution_date`.

`execution_date`

The `execution_date` is the **logical** date and time which the DAG Run, and its task instances, are **running** for.

This allows task instances to process data for the desired logical date & time.

Note: While a task_instance or DAG run might have a *physical* start date of now, their *logical* date might be 3 months ago because we are busy reloading something.

Task

A Task defines a **unit of work** within a **DAG**; it is represented as a **node** in the **DAG graph**, and it is written in Python.

Each **task** is an **implementation of an Operator**, for example a **`PythonOperator`** to execute some Python code, or a **`BashOperator`** to run a Bash command.

The task implements an operator by defining specific values for that operator, such as a Python callable in the case of **`PythonOperator`** or a Bash command in the case of **`BashOperator.`**

```
t1 = BashOperator(
    task_id='print_date',
    bash_command='date',
    dag=dag,
)
```

Task with a BashOperator

Relations between Tasks

Consider the following DAG with two tasks. Each task is a node in our DAG, and there is a dependency from task_1 to task_2:

```
with DAG('my_dag', start_date=datetime(2016, 1, 1)) as dag:
    task_1 = DummyOperator('task_1')
    task_2 = DummyOperator('task_2')
    task_1 >> task_2 # Define dependencies
```

In the above DAG, task_1 is **upstream** of task_2, and conversely task_2 is **downstream** of task_1.

When the **DAG** Run is **created**, **task_1** will **start running** and **task_2 waits** for task_1 to complete successfully before it may start.

Operators

While DAGs describe **how** to run a workflow, **Operators** determine what **actually gets done** by a **task.**

An operator describes a **single task** in a workflow.

Operators are usually (but not always) **atomic**, meaning they can stand on their own and don't need to share resources with any other operators.

The DAG will make sure that **operators** run in the **correct order**; other than those dependencies, operators generally run **independently**.

```
t1 = BashOperator(
    task_id='print_date',
    bash_command='date',
    dag=dag,
)
```

An example BashOperator

There are 3 main types of operators:

- Operators that perform an **action**, or tell another system to perform an action.
- **Transfer** operators move data from one system to another.
- **Sensors** are a certain type of operator that will keep running until a certain criterion is met.

Types of Operators

Airflow provides operators for many common tasks, including:

- **`BashOperator`** - executes a bash command
- **`PythonOperator`** - calls an arbitrary Python function
- **`EmailOperator`** - sends an email
- **`SimpleHttpOperator`** - sends an HTTP request
- ...and many more

Google Cloud Operators:

- BigQuery Operators
- Cloud Functions Operators
- Cloud Storage Operators
- Cloud Transfer Service Operators
- Cloud DataProc Operators
- Cloud Dataflow Operators
- Cloud Build Operators
- Compute Engine Operators



PythonOperator

Use the PythonOperator to run arbitrary Python code.

Cloud Composer runs the Python code in a container that includes the following packages:

- google-cloud-bigquery
- google-cloud-dataflow
- google-cloud-storage



Proprietary + Confidential

01

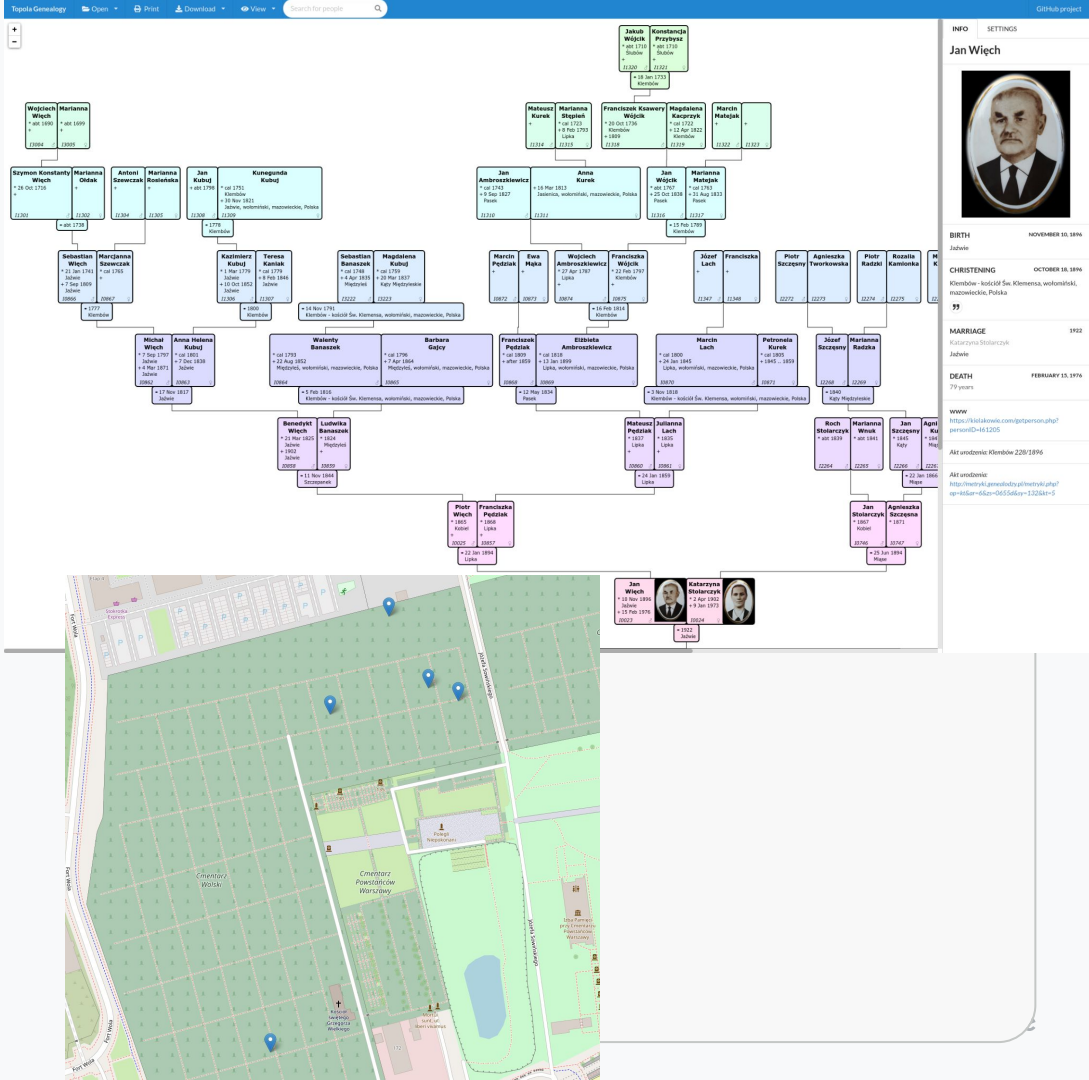
Personal example

My workflow

1. **Edit data**
   - Enter data in GenealogyJ https://sourceforge.net/projects/genj/
   - Save data in GEDCOM format
   - Post-process data with scripts
   - Commit changes to GitLab

2. **Generate and publish artefacts to a web server**
   - Calendar data with birthdays and anniversaries
   - Static HTML files for quick browsing
   - Cemetery maps with marked graves

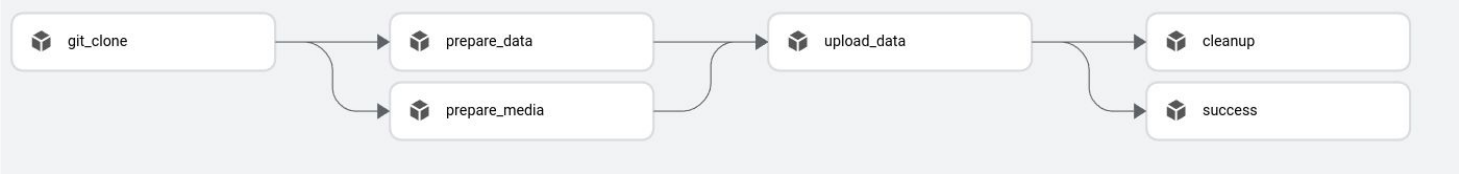
(2) is scripts and a web server running on a VM



Goal: automation in Composer

Setup:

- Composer 2
- DAG with 6 tasks
  - Clone git repository with data
  - Prepare files to be published (2 parallel tasks)
  - Publish data (copy to a GCS bucket)
  - Clean up (remove temp files from GCS `/data` folder)
  - One dummy node
- DAG is manually triggered (could be easily run e.g. once a day)




```
import datetime
import os
import re

from airflow.models import DAG
from airflow.models.variable import Variable
from airflow.operators.bash_operator import BashOperator
from airflow.operators.dummy import DummyOperator
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.operators.gcs import GCSSynchronizeBucketsOperator
from airflow.utils.trigger_rule import TriggerRule

SOURCE_BUCKET = 'us-central1-pwiech-testing-bfeff550-bucket'

TARGET_BUCKET = [redacted]
# TARGET_DIR = [redacted]
TARGET_DIR = 'b[redacted]

REPO_URL = 'git@gitlab.com:[redacted]


default_args = {
    'start_date': datetime.datetime(2023, 10, 3),
}

git_clone_bash = """
    set -e
    PWD=$(pwd)

    # Prepare ssh with proper key to use with git.
    echo "$PRIVATE_KEY" > $PWD/id_rsa
    chmod 0600 $PWD/id_rsa
    echo "ssh -i $PWD/id_rsa -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no \\$*" > $PWD/ssh
    chmod +x $PWD/ssh
    export GIT_SSH="$PWD/ssh"

    # Clone repo and delete the .git folder
    git clone $REPO_URL repo
    rm -rf repo/.git

    # Copy repo to data folder so that it is uploaded to GCS.
    DATA_DIR=/home/airflow/gcs/data/$RUN_ID
    mkdir -p $DATA_DIR
    rm -rf $DATA_DIR/repo
    mv repo $DATA_DIR
"""

prepare_media_bash = """
    set -e
    DATA_DIR=/home/airflow/gcs/data/$RUN_ID
    mkdir -p $DATA_DIR/out
    cp -f $DATA_DIR/repo/zdjecia $DATA_DIR/out/
"""

cleanup_bash = """
    rm -rf /home/airflow/gcs/data/$RUN_ID
"""

def prepare_data_task(**kwargs):
    run_id = kwargs['dag_run'].run_id
    data_dir = f'/home/airflow/gcs/data/{run_id}'

    data = open(f'{data_dir}/repo/all.ged').read()
    data = re.sub(
        'FILE zdjecia/(.*)\.jpg',
        f'FILE https://storage.googleapis.com/{TARGET_BUCKET}/{TARGET_DIR}zdjecia/\\1.jpg',
        data)
    os.mkdir(f'{data_dir}/out')
    with open(f'{data_dir}/out/all.ged', 'w') as f:
        f.write(data)


with DAG('genealogy',
         default_args=default_args,
         schedule_interval=None) as dag:

    git_clone = BashOperator(
        task_id='git_clone',
        bash_command=git_clone_bash,
        env={
            'PRIVATE_KEY': Variable.get('ssh-key'),
            'REPO_URL': REPO_URL,
            'RUN_ID': '{{ run_id }}'
        })

    prepare_media = BashOperator(
        task_id='prepare_media',
        bash_command=prepare_media_bash,
        env={
            'RUN_ID': '{{ run_id }}',
        })

    prepare_data = PythonOperator(
        task_id='prepare_data',
        python_callable=prepare_data_task,
        provide_context=True,
    )

    upload_data = GCSSynchronizeBucketsOperator(
        task_id='upload_data',
        source_bucket=SOURCE_BUCKET,
        source_object='data/{{ run_id }}/out/',
        destination_bucket=TARGET_BUCKET,
        destination_object=TARGET_DIR,
        allow_overwrite=True,
        delete_extra_files=True,
    )

    cleanup = BashOperator(
        task_id='cleanup',
        trigger_rule=TriggerRule.ALL_DONE,
        bash_command=cleanup_bash,
        env={
            'RUN_ID': '{{ run_id }}'
        })

    success = DummyOperator(
        task_id='success'
    )

    git_clone >> prepare_data >> upload_data
    git_clone >> prepare_media >> upload_data
    upload_data >> cleanup
    upload_data >> success
```

Proprietary + Confidential

Problems I needed to solve



How do I clone a git repo?

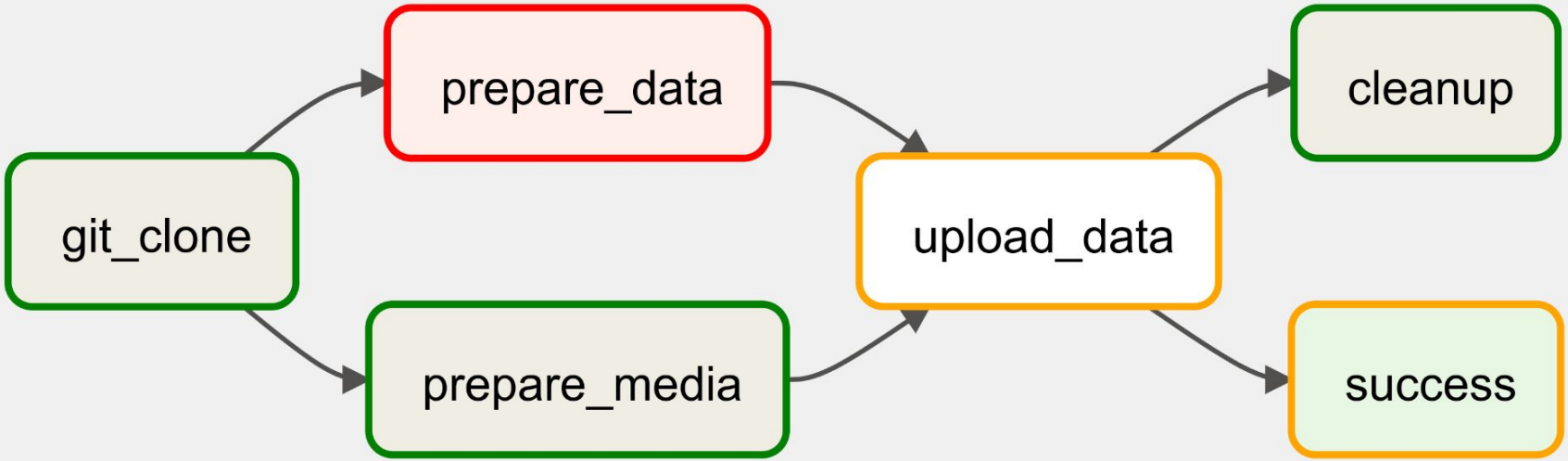
How do I put the private key in Secret Manager?

Where do I clone the repo to?

How do I copy files to GCS?

Proprietary + Confidential

Special “success” `EmptyOperator` task



Google

Summary

- Cloud Composer is a managed Apache Airflow service
  - Simplifies deployment and maintenance
- Schedule and run workflows defined as DAGs in Python code
- Use a variety of operators as tasks in a workflow



Thank you!

https://cloud.google.com/composer
https://github.com/apache/airflow

https://airflowsummit.org/sessions/2023/



pwiech@google.com

Google Cloud